

Projet de programmation

Ermance Lancrenon et Quentin Brisemur

2 avril 2025

1 Introduction :

Notre rendu comporte quatre fichiers python : `grid`, `solver`, `jeu_interactif` et `jeu_interactif constantes` et de nombreux autres fichiers (png et wav) qui servent pour l'interface pygame. Nous n'avons pas traité la question dans laquelle les cases blanches peuvent être appariées avec n'importe quelle case.

1.1 Grid

Le fichier `grid` contient la classe `Grid` modifiée selon les consignes.

1.2 Solver

Le fichier `Solver` contient de nombreuses classes.

La classe `Solver` était fournie dans le fichier de départ.

La classe `SolverGreedy` hérite de la classe `Solver` et correspond à l'algorithme glouton et naïf qu'il nous est demandé d'implémenter.

Il y a ensuite la classe `SolverMatching` qui correspond à la résolution du problème quand toutes les cases valent 1 et utilise la librairie `networkx`.

La classe `SolverMatching_sans_networkx` est un algorithme glouton qui remplace `SolverMatching` mais sans utiliser `networkx`. Puis vient `SolverOptimal_glouton` qui résout le problème dans le cas général avec un algorithme très glouton.

Et enfin la classe `SolverHongrois` résout le problème dans le cas général et en un temps de calcul raisonnable.

1.3 jeu_interactif et jeu_interactif constantes

Ces deux fichiers sont accompagnés de plusieurs fichiers png et wav qu'il est nécessaire de conserver dans le même dossier pour que les importations se passent bien. `jeu_interactif` contient la boucle while de l'interface pygame tandis que `jeu_interactif constantes` contient toutes les fonctions et les constantes nécessaires au programme.

1.4 Divers :

Nos fonctions de score ne font pas systématiquement l'objet d'une longue description. Elles sont toutes construites selon le même principe : on crée une liste des cases qui seront dans le matching optimal. On ajoute le score des cases qui ne sont pas dans cette liste. On ajoute le score de toutes les paires du matching.

Ce document est relativement court et n'entre pas dans les détails les plus techniques. Chaque fonction de nos fichiers contient une documentation précise dans son script.

Ce document a été tapé au clavier et il est très long de mettre un antislash devant chaque underscore pour le faire apparaître sur le pdf, nous espérons que le lecteur saura accorder à cette performance la considération qu'elle mérite.

2 Séance 1 :

Nous ne nous attarderons pas sur le contenu des méthodes `is_forbidden` et `cost` dont la seule lecture du code suffit à la compréhension.

La méthode `plot` est intéressante mais elle est largement surpassée par l'interface créée grâce à pygame. Nous avons décidé de la laisser dans le rendu final mais sans plus d'explications.

2.1 Méthode `all_pairs` :

Une paire étant nécessairement composée de deux cases adjacentes placées sur la même ligne (paire horizontale) ou sur la même colonne (paire verticale), chaque paire contient une case que l'on pourrait appeler « case de gauche » ou « case du dessus », on constate qu'on peut construire la liste de toutes les paires possibles (indépendamment des couleurs) en prenant pour chaque colonne les $n-1$ premières cases et en appelant paire le couple obtenu avec cette case et celle directement en-dessous et en prenant pour chaque ligne les $m-1$ premières cases et en appelant paire le couple obtenu avec cette case et celle directement à sa droite. Une fois cette grande liste de paires possibles obtenue, il ne reste plus qu'à éliminer celles qui ne respectent pas les contraintes de couleurs.

`all_pairs` renvoie donc la liste des couples de cases adjacentes et de couleurs compatibles.

Pour cela, nous avons créé la méthode `color_friendly` qui prend en argument une paire de cases et renvoie True si les couleurs des deux cases sont compatibles, False sinon. Pour cela, on considère chaque cas un à un : d'abord les cas où l'une des cellules est noire, puis l'une est blanche, puis l'une est rouge ou bleue et enfin le cas d'une cellule verte.

2.2 Classe SolverGlouton :

La consigne était de créer un algorithme naïf et glouton. Pour cela, nous avons considéré qu'une méthode de résolution du problème non-optimale mais plutôt intuitive était de sélectionner une case, l'apparier à la paire de coût la plus faible la contenant, supprimer toutes les autres paires contenant l'une des deux cases de la paire choisie et de recommencer avec la case suivante, jusqu'à ce que l'on ne puisse plus faire de paires. Ainsi on prend à chaque fois des paires dont le coût est le plus faible mais cela n'est pas optimal car on ne prend pas en considération le fait que choisir une paire doit être fait en connaissance des modifications du score que cela implique.

La méthode `choix_de_paire_pour_case_donnee` prend en argument une grille et une case et renvoie la (ou l'une des) paire(s) de la grille contenant la case de score le plus faible.

La méthode `choix_glouton_des_paires` prend une grille en argument et renvoie un ensemble de paires dont chacune d'entre elle a été obtenu grâce à la méthode `choix_de_paire_pour_case_donnee`.

Une fois que les paires que l'on va prendre ont été choisies, la méthode `choix_glouton_des_cases_seules` permet de récupérer toutes les cases non-noires qui ne sont choisies dans aucune case.

Finalement, la méthode `score_final_glouton` calcule le score de la grille pour l'appariement choisie avec les méthodes précédentes.

Il est clairement possible de trouver le résultat optimal de cette façon, mais cela dépend entièrement de la grille. Et cela ne fonctionne pour aucune des grilles fournies.

Etant donnée une grille, la complexité de notre algorithme est de l'ordre de 2^{n*m} dans la mesure où la méthode de résolution procède case par case (à la première case nous associons la case qui minimise le coût de la paire, à la deuxième case nous associons...).

3 Séance 2 :

Lors de la résolution du problème dans le cas où la grille ne comporte que des 1, nous avons, comme tout le monde, été confrontés à l'algorithme de Ford-Fulkerson. Pour commencer, nous avons décidé d'utiliser la librairie `networkx` qui contient des fonctions nous permettant de facilement contourner certaines difficultés. Nous avons donc commencé par créer la structure de classe nécessaire à la résolution du problème en construisant notre classe `SolverMatching` autour des fonctions de `networkx` avec le projet de les remplacer plus tard. C'est pourquoi notre solution comprend deux parties : la classe `SolverMatching` qui résout le problème grâce à `networkx` et la classe `SolverMatching_sans_networkx`.

3.1 Classe SolverMatching :

Comme vu en cours, nous avons longtemps été mis en difficulté par la fonction `nx.bipartite.maximum_matching` qui prend en argument un graphe biparti et une liste de sommets censée représenter les sommets d'un côté. Nous avons contourné ce problème en appliquant la fonction uniquement à des graphes connexes. Heureusement, nous avons finalement réussi à la lever et notre programme n'est plus alourdi d'histoires de graphes connexes.

La méthode `graphe_biparti` sert à construire un graphe biparti sur lequel nous allons utiliser des fonctions `networkx`. Nous renvoyons le graphe et la liste des sommets que nous considérons être sur la gauche (les sommets pairs), de laquelle nous pouvons déduire la liste des sommets sur la droite. La méthode `liste_des_paires_choisies` consiste entièrement en l'appel de la fonction `bipartite.maximum_matching` de `networkx` qui prend en argument un graphe biparti et une la liste des sommets sur la gauche du graphe biparti. Elle renvoie un dictionnaire représentant le matching optimal en associant à chaque nœud du matching sous forme de clé, la case lui correspondant sous forme d'attribut. Enfin, la méthode `solution_optimale` calcule le score final. Pour chaque paire de cases choisie grâce à la méthode `liste_des_paires_choisies`, le score augmente de 0 donc il suffit de compter le nombre de cases présentes dans aucune paire (car elles contribuent toutes de 1 au score).

3.2 Classe SolverMatching_sans_networkx :

Cette classe est essentiellement une copie de la classe `SolverOptimal_glouton`, c'est pourquoi nous ne nous attarderons pas dessus ici. La seule différence avec la classe `SolverOptimal_glouton` est le calcul du score. Etant donnée une liste de paire, comme cette classe est censée n'être utilisée que sur des grilles ne contenant que des 1, le score peut-être calculé à partir du nombre de cases non-assignées à une paire. C'est-à-dire que le programme se contente de considérer l'ensemble des paires, de créer une liste des cases utilisées et d'incrémenter le score de 1 pour toute case non comprise dans une paire et non-noire.

4 Séances 3 et 4 :

Notre solution est encore une fois construite en deux parties. La première est un autre algorithme glouton qui, pour une grille donnée, teste toutes les combinaisons de paires possibles et sélectionne celle qui minimise le score. Il ne fonctionne plus en un temps raisonnable à partir de la grille 17. C'est pourquoi notre deuxième piste a été l'algorithme hongrois.

4.1 Classe SolverOptimal_glouton :

`grid.all_pairs()` est l'ensemble des paires possibles. Ainsi, un matching optimal n'est qu'un ensemble de paires de `grid.all_pairs()`. Résoudre le problème peut donc se faire en testant tous les matchings possibles et choisissant celui qui minimise le score. Pour cela, on peut donc voir un matching comme une partie à k éléments de `grid.all_pairs()`.

La méthode `score_d'une_combinaison_de_paires` prend en argument une combinaison de paires sous forme de liste et renvoie le score associé à cette combinaison. La méthode `vérifier_répétition` sert à nous faire gagner un peu de temps de calcul. En effet, parmi les parties de `grid.all_pairs()`, certaines contiennent plusieurs paires ayant une même case en commun. Un tel matching est impossible puisqu'une case ne peut être prise qu'au plus dans une paire. Il ne sert à rien de calculer le score de ces matchings invalides.

La méthode `calcul_du_score` est, comme son nom l'indique, celle qui calcule le score optimal. Pour cela, on définit une variable x qui jouera le rôle de minimum et on définit dans notre méthode la fonction `liste_parties_a_r_elements` qui, étant donnés une liste L et un entier r , renvoie une liste des sous-listes à r éléments de L . Ainsi, `calcul_du_score` calcule le score de toutes les parties à r éléments de `grid.all_pairs()` pour r allant de 0 à la taille de `grid.all_pairs()` et x prend la valeur du minimum des scores obtenus.

Nous avons bien conscience que cette façon de résoudre le problème est inélégante au possible c'est pourquoi nous avons également implémenté la classe `SolverHongrois` qui suit.

4.2 Classe SolverHongrois :

Cette classe est celle qui permet de résoudre le problème dans le cas général et en un temps acceptable. Elle reprend l'algorithme hongrois.

Méthode `construire_matrice` : Cette méthode est celle qui construit la fameuse matrice de coûts nécessaire à la résolution du problème. Les lignes de cette matrice représentent les cases paires et les colonnes les cases impaires. Les numéros de cette matrice valent le coût de l'arête entre les cases associées à la ligne et la colonne du numéro si une telle arête existe, la somme des valeurs des deux cases sinon pour s'assurer que le solver ne renvoie pas une solution qui comprend une arête inexistante.

La méthode `résolution_optimale_avec_algorithme_optimal` est la clé de voute de notre programme. Grâce à la fonction `linear_sum_assignment` de `scipy`, elle renvoie le score et la liste des paires choisies pour le matching optimal.

Le score de la méthode résoudre est renvoyé grâce à la méthode `calculer_score`.

Le méthode `adjacence` n'est là que pour contrôler que l'on ajoute uniquement les paires du matching à `self.pair_trouvees` dans la méthode `résolution_optimale_avec_algorithme_optimal`.

L'algorithme hongrois est censé fonctionner avec une complexité $O(n^4)$ où n est la racine carrée du nombre total de cases. Etant donnée que le notre est employé au sein de la classe `SolverHongrois` et qu'il y a le calcul du score, notre algorithme doit fonctionner avec une complexité $O((n \times m)^{\frac{5}{2}})$.

5 Séance 5 :

Pour cette séance, nous avons choisi d'explorer la piste du jeu interactif dans lequel il faut choisir une paire de cases et un algorithme qui en choisit ensuite une de manière optimale.

Notre programme se décompose sur deux fichiers : `jeu_interactif` qui contient la boucle de jeu et `jeu_interactif constantes` qui contient les fonctions et les variables utilisées dans le jeu.

Notre programme procède de la manière suivante : le joueur humain commence, il sélectionne une case avec un clic gauche, cette case devient orange en attendant d'être appariée à une autre, le joueur en sélectionne une deuxième, si cela crée une paire valide, les deux cases deviennent roses et contribueront au score final en tant que paire. C'est ensuite à l'algorithme de sélectionner une paire, les deux cases deviennent marron. Le jeu s'arrête lorsqu'il n'est plus possible de faire une paire de plus et le score est renvoyé.

5.1 Fichier `jeu_interactif constantes` :

La première étape est la construction de la grille. Les lignes 26 à 42 permettent d'initialisation les objets qui vont nous permettre de la construire. Les cases sont créées à partir de cases en format png de la couleur correspondante. La fenêtre est de taille 1250X800 et la taille des cases est adaptée en fonction de la taille de la grille.

Les trois fonctions d'affichage affichent respectivement la grille, les chiffres et le score (calculé avec la fonction `score_grille`). Il est important de noter que ces trois fonctions ne peuvent afficher vraiment le score qu'une fois que la ligne d'actualisation de la fonction `pygame.display.flip()` aura été lue dans la boucle.

5.2 Fichier `jeu_interactif` :

Celui-ci est donc le fichier qui contient la boucle while du jeu. On commence par afficher la grille au début et on rentre dans la boucle. Deux instructions possibles : fermer la fenêtre et clic gauche. A partir du clic gauche, si l'on clique sur une case noire, le programme renvoie un message d'erreur et il faut choisir une autre case. Si une case de couleur valide est choisie, elle devient orange et on continue. Si une deuxième case est choisie, le programme vérifie que l'on a bien affaire à une paire valide et les deux cases deviennent violettes, sinon il renvoie de nouveau un message d'erreur et redonne leur couleur initiale aux cases. C'est ensuite à l'algorithme de choisir une paire, les deux cases deviennent marron. On passe ensuite à l'affichage du score.

Pour afficher l'actualisation du score et des cases, on superpose à chaque fois des rectangles de couleur. Par exemple si une case est verte, puis sélectionnée (donc orange) puis acceptée dans une paire (donc violette), on met un rectangle vert, on rajoute un rectangle orange par-dessus et ensuite un violet. L'ordre dans lequel l'algorithme exécute ces tâches est donc très important. Pour le score, on l'affiche et quand il change on met un rectangle noir par-dessus avant d'écrire le nouveau score en blanc sur le rectangle noir. Quand une case est intégrée dans une paire valide, sa valeur devient 4 pour indiquer à l'algorithme qu'elle ne peut plus être choisie, il va la traiter comme une case noire.

Une question demeure irrésolue : celle d'arriver à faire jouer l'algorithme de manière optimale. En effet, pour qu'il joue de manière optimale, il faudrait à la fois faire en sorte qu'il choisisse des paires dont le coût permet d'éviter de laisser seules des cases qui contribueraient beaucoup au score (par exemple choisir une paire de coût 4 avec des paires de valeurs 10 et 14 plutôt qu'une paire de coût 4 avec des cases de valeurs 1 et 5) et qu'en même temps il joue de façon à anticiper les mauvais coups que l'on pourrait faire en choisissant volontairement des paires à coût élevé pour l'empêcher de minimiser le score. Il faudrait sûrement s'inspirer de DeepBlue, l'algorithme qui joue aux échecs.

Nous n'avons pas réussi à triompher de manière certaine de cette difficulté. Toutefois, notre algorithme choisit à chaque fois la paire de coût le plus faible, ce qui est toujours mieux que rien !

Ermance Lancrenon - Quentin Brisemur